Formal Verification
Final Report

Polymarket V2 – High Priority Contracts

August 2026
Prepared for Polymarket

certora

Table of contents

certora

Project Summary

Project Scope

Project Name	Repository (link)	Commit Hash	Platform
Polymarket V2	https://github.com/Polymarket/polymarket-v2	Initial: c3be835 Latest: c0df065	EVM

Project Overview

This document describes the specification and verification of **Polymarket** using the Certora Prover and manual code review findings. The work was undertaken from **June 8th** to **August 7th**

The following contract list is included in our scope:

```
{
      src/positionManager/PositionManager.sol
      src/collateral/CollateralToken.sol
      src/modules/BinaryModule.sol
      src/modules/NegRiskModule.sol
      src/modules/CombinatorialModule.sol
      src/bridge/CcipBridge.sol
      src/exchange/Exchange.sol
      src/oracle/OracleAggregator.sol
      src/oracle/modules/OOReporterModule.sol
}
```

The Certora Prover demonstrated that the implementation of the **Solidity** contracts above is correct with respect to the formal rules written by the Certora team. **During the verification process, the Certora team discovered a bug in the Solidity contracts code, as listed on the following pages.**

Protocol Overview

Polymarket V2 is a second generation prediction-market system. It is made of multiple components, including an ERC1155 position ledger (`PositionManager`), a collateral token (pUSD), pluggable market modules, a modular oracle, an exchange, and Chainlink CCIP bridging. The `PositionManager` itself holds no market logic, as it delegates everything to modules, the `OracleAggregator` reports results into them, the `Exchange` matches orders with direct-to-module transfers, and `CcipBridge` moves positions, collateral, and results across chains.

The three modules share the ledger and collateral but differ in what a condition means. `BinaryModule` has one condition with YES/NO outcomes, and its resolution arrives as a payout pair normalized to 1_000_000.
`NegRiskModule` groups many Binary conditions under an event, including a synthetic Other. Remaining conditions derive NO once any condition resolves YES, and Other derives YES once all real conditions resolve NO.
`CombinatorialModule` allows conjunctions of up to 50 Binary or NegRisk legs where YES pays the product of leg payouts and NO the complement.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

certora

Threat and Security Overview

Polymarket V2 is a system whose solvency goal is a single inequality. The protocol mints ERC-1155 position tokens against pUSD collateral and promises to redeem each one at resolution: in every reachable state the collateral the system holds is at least the pUSD it has issued plus the worst-case redemption obligation of every outstanding position, taken over every possible market resolution.

$$\text{assets} \;>=\; \text{pUSD.totalSupply()} \;+\; \text{liabilities}$$

where assets is the sum of the `USDC` and `USDC.e` balances of the vault and the collateral in the old `ConditionalTokens` contract; liabilities is the worst-case redemption scenario for all events in each module. The security goal is asymmetric. Polymarket's operators, resolvers, and bridge roles are **trusted**. They sequence order matching, deliver oracle results, and relay cross-chain messages. Every market participant is not: makers, takers, splitters, redeemers, migrators, and arbitrary callers must be unable to break the inequality, individually or by composing operations. The verification is therefore aimed almost entirely at what an untrusted caller can reach.

Security relies on five mechanisms:

1. **Pre-transfer discipline.** Every core entry point acts on its own balance. Inputs must already have been transferred in so a caller's authority is exactly what they funded. There are no callbacks and no allowance pulls to redirect.
2. **Payout normalization and rounding direction.** Every stored result is length 2 and sums to `RESULT_DENOMINATOR`, YES rounds down and NO takes the complement, so a complementary pair never redeems for more than it was funded with.
3. **The resolution state machine.** Monotone status with three transitions, `Resolved` is terminal, quorum or explicit authority required, and exactly one write to the settlement target per request.
4. **Role partitions and pause switches.** Owner / admin / operator / resolver / bridge / minter / wrapper / rule-manager are separated per contract, with dedicated pausing mechanisms.
5. **Bridge symmetry.** Value is destroyed only alongside an outbound message and created only by a peer-authenticated `ccipReceive`.

Roles & Trust Boundaries

Role	Trust Level	Powers
Proxy Owner (per contract)	Highest	UUPS upgrade of every proxy; grant admins; `CollateralToken` minter/wrapper sets; `CcipBridge` peers and per-module support flags
PositionManager Admin	Highest	Register and remove modules; set `crossModuleAuth` i.e. decide who may mint and burn any position
CollateralToken Minter	Highest	Mint and burn pUSD, direct control of total supply
Module Admin	High	Grant/revoke resolver and bridge roles; pause a resolver address; pause resolution for an event
Resolver	High	Report results for conditions
Bridge	High	`mintFromBridge` / `burnFromBridge`; relay results, including the neg-risk synthetic Other and migration conditions
Oracle Operator	High	`initializeRequest`; edit a request's reporter/disputer modules, arbitrator, finalizer, liveness; `updateRequestRules`; pause markets
Oracle Admin	High	Manage operators and rule managers; global pause; `resolveResult` override; all operator actions except `initializeRequest`

Exchange Admin	High	Manage operators; pause trading; set fee receiver and max fee rate; pause/unpause users
Exchange Operator	High	Match orders, the sole entry point to settlement; pre-approve and invalidate orders
Reporter / Disputer / Arbitrator modules	High	Cast votes, raise disputes, trigger arbitration and resolve. Membership is set by the Oracle Operator
CollateralToken Wrapper	Medium	Wrap / Unwrap against the vault, for whitelisted assets only
Migration Creator / Operator	Medium	Prepare migration conditions and events; batch-migrate legacy positions on behalf of users
Market participant / anyone	Untrusted	Split, merge, redeem, convert, horizontal and combinatorial operations, `prepareCondition`, wrap/unwrap via routers, sign orders, self-pause, relay a settled UMA price, finalize when the finalizer is unset

Primary trust boundaries. (1) **Participant vs collateral** the pre-transfer boundary. A module acts only on its own balance, so a caller can never cause an operation larger than what they funded. (2) **Module vs ledger** only the registered module for a position id, or one holding `crossModuleAuth`, can change its supply, the registry itself is admin-only and cross-auth is cleared on removal. (3) **Resolver vs module** resolution is authorized by role and scoped by the `resolutionChain` encoded in each condition ID, and a stored result is immutable thereafter. (4) **Operator vs maker** the operator prices and sequences settlement, but every fill still requires that maker's own signature or an active preapproval. (5) **Source chain vs destination chain** a destination mint is authorized by peer identity alone, and `mintFromBridge` intentionally breaks

the local solvency inequality because its backing is a burn on the remote chain. Global cross-chain solvency therefore rests on peer honesty and correct peer configuration. The same holds for the legacy `ConditionalTokens`: its payout numerators are trusted as reported.

Expected Invariants

1. **Global solvency.** Collateral held is always at least pUSD issued plus the worst-case redemption obligation of all outstanding positions. Equivalently, every module operation satisfies Δ pUSD.totalSupply() + Δ L ≤ 0, with the liability L evaluated over every possible resolution.
2. **Value conservation of position transforms.** No reshuffling operation creates redeemable value: forward refinements mint no more than they burn at every resolution, wrap / unwrap are exact equalities, refinement round trips are exactly neutral, and compress yields collateral plus a residual worth no more than its input. The inverse direction is the one non-strict case as it may recover at most 1 wei of floor-rounding dust per operation, covered by the surplus retained when the parts were created.
3. **Payout normalization and safe rounding.** Every stored result has length 2 and sums to `RESULT_DENOMINATOR`. Redeeming the YES and NO sides of the same amount never returns more than that amount, is exact when divisible, and loses at most one unit otherwise.
4. **Result immutability.** Once a condition's result is stored it never changes under any method. A report carrying a different payout vector always reverts.
5. **Neg-risk partition completeness.** At most one condition per event resolves YES. `conditionsResolved` is monotone and bounded by arity, and once the event is decided the derived sibling-NO and synthetic-Other results complete the partition to exactly `RESULT_DENOMINATOR`.
6. **Ledger authority.** Position supply changes only through the registered module for that position id or a `crossModuleAuth` module; `crossModuleAuth` implies prior registration and a module always self-reports its own id.
7. **Module escrow.** No module function increases the module's own pUSD balance, or its balance of any position id, beyond what the caller explicitly directed to it.
8. **Order consent and fill monotonicity.** No order fills without a signature binding its maker or an active preapproval, and preapproval is reachable only through the signed path. Cumulative fills never exceed `makerAmount`, overfill reverts, and the filled latch is never cleared.

9. **Settlement conservation and fee bounds.** `matchOrders` neither mints nor drains collateral: value in equals value out plus fees, the `Exchange` retains no residual, the taker receives exactly the operator-declared amount, and no per-fill fee exceeds the maximum rate or the proceeds it is taken from.
10. **Resolution lifecycle and exactly-once settlement.** Request status is monotone with exactly three transitions and `Resolved` is terminal; a request resolves only via reporter quorum or an authorized override; a live proposal is always backed by quorum; the settlement target is written at most once; the finalized outcome is caller-independent and matches the committed proposal.
11. **Collateral backing and supply authority.** `wrap` and `unwrap` are exactly 1:1 with the vault and touch no other account's balance; only `MINTER_ROLE` can change pUSD supply, only `WRAPPER_ROLE` can wrap or unwrap.
12. **Initialization authority.** Initializers succeed at most once, the version is frozen once non-zero, and disabled implementations remain permanently non-initializable.
13. **Bridge symmetry.** A burn on the source chain is followed by an equivalent mint on the destination. Value is destroyed only alongside an outbound message, created only by `ccipReceive` from a registered peer, and a bridged result moves no value at all.

Attack Vectors

1. Unbacked minting and solvency drain

A participant chains split, refinement, conversion, compression, merge, and redeem to finish holding more collateral or more redeemable value than they funded, or exploits rounding dust at scale.

Key properties

- assets >= pUSD.totalSupply() + liabilities (global solvency)
- forward refinements (`splitOnCondition`, `extract`, `convertToYesBasket`): minted value ≤ burned value at every resolution
- inverse refinements (`mergeOnCondition`, `inject`, `mergeFromYesBasket`): minted value ≤ burned value + 1 wei, and the forward-then-inverse round trip is exactly neutral
- `wrap` / `unwrap`: exact value equality
- `compress`: collateral out plus residual value ≤ input value, including the terminal and all-resolved cases

- position-only transforms are collateral-neutral; modules retain zero pUSD and zero positions between operations

2. Forged or misrouted position minting

An attacker calls the ledger directly without the required roles, or routes through a module that does not own the position id.

Key properties

- `mint`, `burn`, `batchMint`, `batchBurn` revert unless the caller is the registered module for every position id supplied, or holds `crossModuleAuth`
- balance changes are reachable only through those authorized entry points
- `crossModuleAuth` requires prior registration, is admin-only, and is cleared when a module is removed

3. Combinatorial condition collisions

The combinatorial base hash is truncated to 128 bits, so an attacker plants a leg array that collides with a live condition ID and re-prices an outstanding position or operates against a condition that was never prepared.

Key properties

- `prepareCondition` stores only canonical leg arrays: non-empty, strictly ascending, distinct condition IDs, binary/neg-risk legs with outcome < 2
- a prepared condition stays prepared
- stored legs cannot be changed

4. Collateral ramp abuse

An attacker mints pUSD without the underlying arriving, releases vault assets without burning, or wraps an asset the system never intended to accept.

Key properties

- `wrap` mints exactly the pre-transferred amount and forwards exactly that amount to the vault
- `unwrap` releases exactly the requested amount to the recipient and burns it from the contract's own balance
- `wrap` and `unwrap` do not touch any other account's balance
- `mint` / `burn` revert without `MINTER_ROLE`; wrap / unwrap revert without `WRAPPER_ROLE` or for a non-whitelisted asset

5. Unauthorized or contradictory resolution

An unauthorized caller reports a result for a condition or a second report overwrites a stored result with a different payout vector.

Key properties

- a differing payout vector always reverts
- stored results are length 2 and sum to `RESULT_DENOMINATOR`; malformed vectors revert
- a resolved condition is immutable

6. Neg-risk partition corruption

A second condition resolves YES, or the synthetic Other is reported directly, so an event's payouts sum past `RESULT_DENOMINATOR` and the event becomes over-redeemable.

Key properties

- the event's YES aggregate reaches `RESULT_DENOMINATOR` at most once
- `conditionsResolved` is monotone, bounded by arity, and matches the count of stored results
- full resolution implies the partition sums to exactly `RESULT_DENOMINATOR`, with siblings deriving NO and Other deriving YES
- `horizontalSplit` / `horizontalMerge` cover exactly arity+1 YES positions with equal amounts and are exact inverses; convert burns one NO and mints YES for every other outcome, conserving notional and moving no collateral

7. Order settlement without consent

An attacker fills a maker's order with no signature, replays a filled order, overfills it, or the settlement returns the taker less than was declared.

Key properties

- a fill requires either an active preapproval or a verifier-accepted signature over the order's own hash; EOA, ERC-1271, proxy, and Safe signature types each bind the maker
- an empty signature passes if and only if the order is preapproved, and preapproval is reachable only through the signed path
- cumulative fills never exceed `makerAmount`; overfill reverts; the filled latch is never cleared, on any entry point including the combinatorial one
- the taker receives exactly the operator-declared `takerReceiveAmount` on every path
- `matchOrders` neither mints nor drains collateral and leaves no residual in the `Exchange`
- fees never exceed the maximum rate or the proceeds of the fill they are taken from

8. Self-pause bypass and griefing

A user pauses to protect themselves but their orders keep filling; or an attacker pauses someone else, or stretches the pause interval until the delay is meaningless or the exit is bricked.

Key properties

- `pause` and `unpause` are self-service; a caller cannot change another user's pause state
- the pause becomes effective after the configured block interval; a zero interval is immediate
- once effective, neither the user's maker orders nor their taker orders can fill
- the interval is capped, and only admins can set it
- `pause` and `unpause` write exactly the target user's slot and nothing else

9. Resolution manipulation

An attacker votes twice, disputes without a proposal, drives a proposal through below quorum, finalizes twice, or bricks a market.

Key properties

- status is monotone with exactly three transitions and `Resolved` is terminal; the active state is never persisted
- a request becomes `Resolved` only via a quorum-backed finalize or an authorized override
- a live proposal is always backed by reporter quorum, and implies a registered request
- each report or dispute sets one flag and advances one counter by exactly one; a repeat from the same module reverts; a dispute requires a prior proposal; no other method moves either counter
- the target is written only from finalize or resolve, never twice for the same request
- windows freeze once proposed, are set to now-plus-liveness on a fresh proposal, and reject reports and disputes at or after expiry
- the finalized outcome is caller-independent and matches the proposal the hash committed to
- arbitrator hook failures never block threshold escalation, reporter-conflict escalation, or admin resolution

10. Relay result forgery

The relay reports something other than the deterministic translation of the settled price, or an improper value as a payout / index.

Key properties

- the array the aggregator records is exactly the module's deterministic translation of the settled price, and is independent of who relayed it
- the atomic neg-risk path forwards a raw winner index, accepted exactly for non-negative, denominator-aligned indices below arity
- price acceptance is exactly {YES, NO} for every market type, plus `P3` only under `BINARY`; negative prices are rejected
- no method moves ERC-20 value or ETH out of the reporter module, and none grants an allowance

11. Cross-chain value forgery

An attacker delivers a message that mints positions or collateral on the destination with no matching burn on the source, or replays a delivered message.

Key properties

- `bridgePositions` and `bridgeCollateral` burn exactly what the outbound message specifies
- value is destroyed only alongside an outbound message and created only by `ccipReceive`; messages originate only from the three bridge entry points
- `ccipReceive` rejects callers other than the router and messages from unregistered peers or chains
- `bridgeResult` moves no value
- a sent bundle is deliverable at an honestly-configured destination

12. Privilege escalation and upgrade capture

An attacker re-initializes a proxy or an implementation, seizes the upgrade path, or crosses a role boundary.

Key properties

- `initialize` succeeds at most once and requires a fresh slot; the version moves only 0 → 1 and is frozen once non-zero
- module registration, removal, and `crossModuleAuth` are admin-only
- `CollateralToken` role administration and upgrades are owner-only

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	-	-	-
Low	-	-	-
Informational	1	1	1
Total	1	1	1

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
		Likelihood		

Informational Issues

I-01. Users lose positions / collateral when they provide a non-EVM recipient while bridging.

Description: `CcipBridge.bridgePositions` and `CcipBridge.bridgeCollateral` take the recipient address as a bytes32 without checking whether the provided value is a valid EVM address. This causes incoming messages to revert because `_processMessage` tries to decode an invalid address, effectively burning positions or collateral.
This issue has been classified as informational because the lost positions or funds are burned from the caller of the functions, which is also the one losing value. In addition, the caller has to intentionally provide a dirty recipient.

Customer's response: Confirmed, and fixed in PR-306.

certora

Formal Verification

Verification Methodology

We performed verification of the Polymarket V2 protocol using the Certora verification tool which is based on Satisfiability Modulo Theories (SMT). In short, the Certora verification tool works by compiling formal specifications written in the Certora Verification Language (CVL) and Polymarket's implementation source code written in Solidity.
More information about Certora's tooling can be found in the Certora Technology Whitepaper.

If a property is verified with this methodology it means the specification in CVL holds for all possible inputs. However specifications must introduce assumptions to rule out situations which are impossible in realistic scenarios (e.g. to specify the valid range for an input parameter). Additionally, SMT-based verification is notoriously computationally difficult. As a result, we introduce overapproximations (replacing real computations with broader ranges of values) and underapproximations (replacing real computations with fewer values) to make verification feasible.

Rules: A rule is a verification task possibly containing assumptions, calls to the relevant functionality that is symbolically executed and assertions that are verified on any resulting states from the computation.

Inductive Invariants: Inductive invariants are proved by induction on the structure of a smart contract. We use constructors as a base case, and consider all other (relevant) externally callable functions that can change the storage as step cases.
Specifically, to prove the base case, we show that a property holds in any resulting state after a symbolic call to the respective constructor. For proving step cases, we generally assume a state where the invariant holds (induction hypothesis), symbolically execute the functionality under investigation, and prove that after this computation any resulting state satisfies the invariant.

Verification Notations

Formally Verified	The rule is verified for every state of the contract(s), under the assumptions of the scope/requirements in the rule.
Formally Verified After Fix	The rule was violated due to an issue in the code and was successfully verified after fixing the issue
Violated	A counter-example exists that violates one of the assertions of the rule.

General Assumptions and Simplifications

To make formal verification tractable, we may apply techniques that change or replace parts of the original code with equivalent ones or over-approximations. This can happen via multiple ways: (1) Solidity harness contracts inheriting all behavior from the original contract but e.g. exposing private fields or methods, (2) function summaries with CVL functions, or (3) minor patches on the original code.

For Polymarket V2 we override some functions to store additional data in the harness, and then call the underlying functions. This is safe as the harnessed contracts are not inherited by any other, so the inheritance chain is not affected. We achieve this by rendering the original functions virtual. Adding virtual to the original functions does not change their behavior or the call stack in otherwise unchanged code, that is every call site still resolves to the same body. The function bodies themselves are untouched.
In addition, we replaced some assembly blocks with equivalent Solidity code so the Prover can properly perform analysis on them. Then, in order to summarize `getResult` and `storeResult` in CVL consistently, inside the modules we replace direct reads of the `result` mapping with calls to `getResult`. This is safe as `getResult` does not perform any extra operation other than reading from the mapping.

We developed summaries for `PositionManager` as Solady's storage layout makes the storage analysis fail for the Prover. In particular we replaced assembly mutators with a CVL model that is behaviorally equivalent to the original code. For the same reason, we replaced Solady's

`EnumerableSetLib` library with a pure Solidity implementation, where we also proved the correct behavior of the used functions.
We summarized `getPayout` in `CombinatorialModule` with an equivalent model under the 2-leg assumptions. In addition, we over-approximated the legacy payout division to use a nondeterministic result between 0 and 1 million. This is sound because it covers more values compared to the actual code, and at the same time it removes non-linearity in the problem thus making it more tractable for the SMT solvers.
In addition, we use summaries for `mulDiv` operations, `SafeTransferLib` and roles management, which replace the Solady implementations.

When dealing with loops, the Prover unrolls them a fixed number of times. Based on the smart contract under verification and the complexity of the property, we typically limit the number of iterations from 2 to 5. In particular, for `NegRiskModule` solvency we typically limit it to 3, and for `CombinatorialModule` to 2. This is enough to include at least two conditions and the synthetic Other in `NegRiskModule`, and have all possible combinations of underlying positions in `CombinatorialModule` (Binary-Binary, Binary-NegRisk, NegRisk-NegRisk).
We also assume that loops do not run more times than the unroll bound.

Lastly, value ranges might be bounded where the contracts use unchecked arithmetic. `Exchange` order and fill amounts are bounded below 2^120 so the unchecked accumulators and products cannot wrap, and makerAmount below 2^248. These bounds sit far above any economically meaningful amount.

Formal Verification Properties Overview

ID	Title	Impact	Status
P-01	BinaryModule Global Solvency	Holders could redeem more collateral than the module ever took in, leaving outstanding positions unbacked	Verified
P-02	BinaryModule Position Lifecycle	split, merge or redeem could mint or burn the wrong amounts, letting a user withdraw more collateral than they put in	Verified
P-03	BinaryModule Resolution Integrity	A malformed or mutable result could make a condition pay out more than the denominator	Verified
P-04	BinaryModule Bridge Authorization	An unauthorized caller could mint or burn positions, forging value out of nothing	Verified
P-05	NegRiskModule Global Solvency	Holders could redeem more collateral than the module ever took in, leaving outstanding positions unbacked	Verified
P-06	NegRiskModule Position Lifecycle	A neg-risk operation could mint or burn the wrong outcome conditions, so an event would owe its holders more collateral than was ever paid into it	Verified
P-07	NegRiskModule Resolution Integrity	An event partition could exceed the full denominator, so its positions would redeem for more than was committed	Verified
P-08	NegRiskModule Bridge Authorization	An unauthorized caller could mint or burn positions, forging value out of nothing	Verified

P-09	CombinatorialModule Global Solvency	Holders could redeem more collateral than the module ever took in, leaving outstanding positions unbacked	Verified
P-10	CombinatorialModule Position Lifecycle	split, merge or wrap could break the pairing between a conjunction and its complement, so the pair would redeem for more collateral than created it	Verified
P-11	CombinatorialModule Refinement Value Conservation	A refinement could mint positions worth more than those it consumed, creating redemption value from nothing	Verified
P-12	CombinatorialModule Conjunction Store	A condition id could bind to the wrong leg set, settling a position against a different market than it was sold as	Verified
P-13	PositionManager Token Integrity	mint and burn could move amounts other than requested, so position supply would stop matching the collateral behind it	Verified
P-14	PositionManager Solvency	Position supply could move outside the mint and burn entry points, creating unbacked positions	Verified
P-15	PositionManager Access Control	A contract that is not the position module could mint or burn it, forging unbacked positions	Verified
P-16	Exchange Collateral Conservation	A match could mint or drain collateral, or pay a party the wrong amount	Verified
P-17	Exchange Order Authorization	An order could be filled without its maker consent, spending someone else balance	Verified
P-18	Exchange Order Accounting	An order could be overfilled or refilled after completion, draining the maker beyond what	Verified

		they signed	
P-19	Exchange Combinatorial Binding	The prepared condition could differ from the taker token, settling the trade against the wrong market	Verified
P-20	CollateralToken Wrap Backing	pUSD could be minted without the matching asset reaching the vault, leaving the collateral token unbacked	Verified
P-21	CollateralToken Supply Access Control	An unauthorized caller could change pUSD supply, inflating the collateral token	Verified
P-22	OracleAggregator Resolution Lifecycle	A request could resolve twice, resolve without authority, or report to the wrong condition, settling a market incorrectly	Verified
P-23	OracleAggregator Vote Accounting	An outcome could enter the dispute window without quorum, or one reporter could vote repeatedly, settling on an unbacked result	Verified
P-24	OracleAggregator Request Configuration	A request could be left with a zero arbitrator or a result shape the reporters cannot satisfy, making resolution or escalation impossible	Verified
P-25	OracleAggregator Access Control	An unauthorized caller could resolve, report on, or reconfigure a request, dictating a market outcome	Verified
P-26	OOReporterModule Result Translation	A settled price could translate into the wrong payout vector, settling the market on the wrong outcome	Verified
P-27	OOReporterModule Registration and Access Control	A request could be registered twice or by an unauthorized caller, re-pointing a market at a different question	Verified

P-28	CcipBridge Cross-Chain Fidelity	A transfer could mint more than was burned, or be delivered by an untrusted sender, forging value across chains	Verified after fix
P-29	Initialization Access Control	An initialized contract or a bare implementation could be re-initialized, letting an attacker take ownership	Verified

Detailed Properties

BinaryModule Global Solvency

Module General Assumptions

- Loops are unrolled to between 2 and 5 iterations, stated per rule where it is not the upper bound.
- The legacy condition-id helpers are summarized in CVL.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.

Module Properties

BINARY-GLOB-SOLVENCY			
Status: Verified		BinaryModule preserves the collateral backing of every outstanding position it has minted, under every possible market resolution.	
Rule Name	Status	Description	Link to rule report
solvencyPreserved	Verified	*BinaryModule preserves the backing inequality between counted assets and the sum of pUSD supply and worst-case liability.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
solvencyPreservedLengthSmallerOrEqualThan1 (BinaryModuleMigrate)	Verified	*migratePositions preserves the backing inequality for batches of at most one element.* *Assumption: Case split covering batches of at most one element.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report
solvencyPreservedSameCondition (BinaryModuleMigrate)	Verified	*migratePositions preserves the backing inequality when both elements resolve to one condition.* *Assumption: Case split covering two elements resolving to one condition.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report

solvencyPreservedDistinctConditions (BinaryModuleMigrate)	Verified	*migratePositions preserves the backing inequality when the two elements resolve to distinct conditions.* *Assumption: Case split covering two elements resolving to distinct conditions.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report
solvencyPreservedLengthSmallerOrEqualThan1 (BinaryModuleMigrateFrom)	Verified	*The delegated migratePositions preserves the backing inequality for batches of at most one element.* *Assumption: Case split covering batches of at most one element.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report
solvencyPreservedSameCondition (BinaryModuleMigrateFrom)	Verified	*The delegated migratePositions preserves the backing inequality when both elements resolve to one condition.* *Assumption: Case split covering two elements resolving to one condition.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report
solvencyPreservedDistinctConditions (BinaryModuleMigrateFrom)	Verified	*The delegated migratePositions preserves the backing inequality when the two elements resolve to distinct conditions.* *Assumption: Case split covering two elements resolving to distinct conditions.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report

BINARY-MODULE-ESCROW-01	
Status: Verified	No BinaryModule function increases the module's own pUSD balance, or its balance of any position id, beyond what the caller explicitly directed to it.

Rule Name	Status	Description	Link to rule report
moduleNeverAccumulatesCollateral	Verified	*The module's own pUSD balance grows by at most the amount the caller explicitly directed to it.*	Report
moduleNeverAccumulatesPositions	Verified	*The module's own balance of every position id grows by at most the amount the caller explicitly minted to it for that id.*	Report

BINARY-SOLVENCY-02

Status: Verified	Per binary condition, the redemption obligation of the outstanding YES and NO supply never exceeds the collateral committed to that condition.

Rule Name	Status	Description	Link to rule report
preResolutionSolvency	Verified	*Before resolution the outstanding YES and NO supply of a condition is covered by the collateral committed to it.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
mintFromBridgePreResolutionSourceBacked	Verified	*A bridge mint preserves the pre-resolution backing bound for the condition it credits.* *Assumption: Assumes position is collateralized in another chain. Safe as we showed that solvency holds per individual chain, and bridging operations are solvency preserving.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
mintFromBridgePostResolutionSourceBacked	Verified	*A bridge mint preserves the resolved backing bound for the condition it credits.* *Assumption: Assumes position is collateralized in another chain. Safe as we showed that solvency holds per individual chain, and bridging operations are solvency preserving.*	Report

		Assumption: Verified with loops unrolled to at most 3 iterations.	
postResolutionSolvency	Verified	*After resolution the redemption obligation of a condition is covered by the collateral committed to it.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

BINARY-SOLVENCY-03

Status: Verified	a resolved YES and NO pair redeems for the amount that funded it, up to one unit of floor rounding.

Rule Name	Status	Description	Link to rule report
redemptionDoesNotOverpay	Verified	*A YES and NO pair never redeems for more than the collateral that funded it.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
redemptionLosesAtMostOneUnit	Verified	*Floor rounding drops at most one unit of collateral from a redeemed pair.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
redemptionIsExactWhenDivisible	Verified	*When the YES leg divides evenly, redeeming the pair returns exactly the funding amount.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
endToEndSplitResolveRedeem	Verified	*Splitting collateral, reporting an arbitrary valid resolution, then redeeming both legs returns the funding amount up to one unit.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

endToEndMultiOperationSplitMergeResolveRedeem	Verified	*A sequence of splits and merges followed by resolution and redemption still returns no more than the collateral committed.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

BinaryModule Position Lifecycle

Module General Assumptions

- Loops are unrolled to between 3 and 5 iterations, stated per rule where it is not the upper bound.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.

Module Properties

BINARY-MERGE-01			
Status: Verified		BinaryModule merge is the exact inverse of split.	
Rule Name	Status	Description	Link to rule report
mergeBurnsYesAndNoExactly	Verified	*A successful merge burns exactly the requested amount of the YES and NO positions from the module.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
mergeMintsCollateralExactly	Verified	*A successful merge mints exactly the requested amount of pUSD to the recipient.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
mergeMintsOnlyRecipientCollateral	Verified	*A successful merge changes no collateral balance other than the recipient's.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
mergeBurnsOnlyModulePositions	Verified	*A successful merge touches no position balance other than the two legs held by the module.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

BINARY-REDEEM-01

Status: Verified	BinaryModule redeem pays the floor of the resolved payout, bounded by the amount, and reverts on an unresolved condition or an invalid outcome.

Rule Name	Status	Description	Link to rule report
redeemPaysExactFloor	Verified	*A successful redeem mints exactly the floor of the amount scaled by the resolved numerator.*	Report
redeemPayoutNeverExceedsAmount	Verified	*For a resolved condition the payout never exceeds the redeemed amount.*	Report
redeemBurnsPositionExactly	Verified	*A successful redeem burns exactly the redeemed amount from the module and lowers supply by the same.*	Report
redeemBurnsOnlyThatPosition	Verified	*A successful redeem touches no position balance other than the redeemed one.*	Report
redeemMintsOnlyRecipientCollateral	Verified	*A successful redeem changes no collateral balance other than the recipient's.*	Report
redeemRevertsWhenUnresolved	Verified	*redeem reverts when the condition has no stored result.*	Report
redeemRevertsOnBadOutcome	Verified	*redeem reverts when the outcome index is out of range.*	Report

BINARY-SPLIT-01

Status: Verified	BinaryModule split mints equal YES and NO and burns exactly the pre-transferred collateral.

Rule Name	Status	Description	Link to rule report
splitMintsYesAndNoExactly	Verified	*A successful split credits each recipient by exactly the requested amount of the YES and NO positions.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
splitBurnsCollateralExactly	Verified	*A successful split burns exactly the requested amount of pUSD from the module's own balance.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
splitBurnsOnlyModuleCollateral	Verified	*A successful split changes no collateral balance other than the module's.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
splitTouchesOnlySplitPositions	Verified	*A successful split touches no position balance other than the two it credits.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

BinaryModule Resolution Integrity

Module General Assumptions

- Loops are unrolled to at most 5 iterations.

Module Properties

BINARY-RESULT-NORMALIZED-01			
Status: Verified		Every BinaryModule stored result is a complementary pair summing to the result denominator, and reportResult rejects a malformed vector.	
Rule Name	Status	Description	Link to rule report
resultNormalized	Verified	*For every condition with a stored result, the result is a pair summing to the result denominator.*	Report
reportResultRevertsOnMalformed	Verified	*reportResult reverts on any result vector that is not a complementary pair.*	Report
reportResultCannotChangeStoredResult	Verified	*reportResult can never overwrite a result that is already stored.*	Report
payoutConservation	Verified	*The YES and NO payouts of a resolved condition together account for the amount that funded it.*	Report

BinaryModule Bridge Authorization

Module General Assumptions

- Loops are unrolled to at most 5 iterations.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.

Module Properties

BINARY-BRIDGE-02			
Status: Verified		BinaryModule mintFromBridge and burnFromBridge are bridge-role-only and move exactly the tuple they are given.	
Rule Name	Status	Description	Link to rule report
mintFromBridge OnlyBridge	Verified	*Only an address holding the bridge role can mint; without it the call reverts.*	Report
mintFromBridge CreditsRecipient	Verified	*A successful bridge mint credits exactly the requested amount to the recipient and raises supply by the same.*	Report
mintFromBridge NoOtherBalance Change	Verified	*A bridge mint touches no balance other than the credited one.*	Report
burnFromBridge OnlyBridge	Verified	*Only an address holding the bridge role can burn; without it the call reverts.*	Report
burnFromBridge DebitsModule	Verified	*A successful bridge burn debits the module by exactly the total targeting each id and lowers supply by the same.*	Report
burnFromBridge NoOtherAddress Change	Verified	*A bridge burn only ever debits the module; no other holder is touched.*	Report

NegRiskModule Global Solvency

Module General Assumptions

- Loops are unrolled to between 3 and 5 iterations, stated per rule where it is not the upper bound.
- The legacy condition-id helpers are summarized in CVL.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.

Module Properties

NEGRISK-GLOB-SOLVENCY			
Status: Verified		NegRiskModule preserves the collateral backing of every outstanding position it has minted, under every possible market resolution.	
Rule Name	Status	Description	Link to rule report
solvencyPreserved	Verified	*NegRiskModule preserves the backing inequality between counted assets and the sum of pUSD supply and per-event worst-case liability.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
solvencyPreservedConvert	Verified	*convert preserves the backing inequality against the per-event scaled liability.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
solvencyPreservedMigrateLengthSmallerOrEqualThan1	Verified	*migratePositions preserves the backing inequality for batches of at most one element.* *Assumption: Case split covering batches of at most one element.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
solvencyPreservedMigrateSameCondition	Verified	*migratePositions preserves the backing inequality when both elements map to one structured condition.*	Report

		Assumption: Case split covering two elements mapping to one structured condition, the complementary pair the non-strict input ordering admits. *Assumption: Verified with loops unrolled to at most 3 iterations.*	
solvencyPreservedMigrateSameEvent	Verified	*migratePositions preserves the backing inequality when the two elements sit on different conditions of one event.* *Assumption: Case split covering two elements on different conditions of one event, whose supplies couple through the shared event term.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
solvencyPreservedMigrateDistinctEvents	Verified	*migratePositions preserves the backing inequality when the two elements sit on distinct events.* *Assumption: Case split covering two elements on distinct events.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
solvencyPreservedMigrateFromLengthSmallerOrEqualThan1	Verified	*The delegated migratePositions preserves the backing inequality for batches of at most one element.* *Assumption: Case split covering batches of at most one element.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
solvencyPreservedMigrateFromSameCondition	Verified	*The delegated migratePositions preserves the backing inequality when both elements map to one structured condition.* *Assumption: Case split covering two elements mapping to one structured condition, the complementary pair the non-strict input ordering admits.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
solvencyPreservedMigrateFromSameEvent	Verified	*The delegated migratePositions preserves the backing inequality when the two elements sit on different conditions of one event.* *Assumption: Case split covering two elements on different conditions of one event, whose supplies*	Report

		couple through the shared event term. *Assumption: Verified with loops unrolled to at most 3 iterations.*	
solvencyPreservedMigrateFromDistinctEvents	Verified	*The delegated migratePositions preserves the backing inequality when the two elements sit on distinct events.* *Assumption: Case split covering two elements on distinct events.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

NEGRISK-MODULE-ESCROW-01

Status: Verified	No NegRiskModule function increases the module's own pUSD balance, or its balance of any position id, beyond what the caller explicitly directed to it.

Rule Name	Status	Description	Link to rule report
moduleNeverAccumulatesCollateral	Verified	*The module's own pUSD balance grows by at most the amount the caller explicitly directed to it.*	Report
moduleNeverAccumulatesPositions	Verified	*The module's own balance of every position id grows by at most the amount the caller explicitly minted to it for that id.*	Report

NegRiskModule Position Lifecycle

Module General Assumptions

- Loops are unrolled to between 3 and 5 iterations, stated per rule where it is not the upper bound.
- The legacy condition-id helpers are summarized in CVL.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.

Module Properties

NEGRISK-CONVERT-01			
Status: Verified		convert burns one NO leg and mints a YES leg for every other condition of the event, conserving notional.	
Rule Name	Status	Description	Link to rule report
convertMintsEachOtherYesExactly	Verified	*Every other-condition YES leg of the event is credited exactly the converted amount to the recipient.*	Report
convertSkipsSourceYes	Verified	*The source condition's own YES leg is never minted.*	Report
convertBurnsSourceNoExactly	Verified	*Exactly the converted amount of the source NO leg is burned from the module.*	Report
convertNoCollateralChange	Verified	*convert changes no collateral balance and no collateral supply.*	Report
convertDoesNotTouchOtherNo	Verified	*Only the source NO leg is burned; any other condition's NO leg is untouched.*	Report

convertDoesNotTouchBeyondOther	Verified	*The YES basket stops at the synthetic Other condition; the leg one past it is untouched.*	Report
convertCreditsOnlyRecipient	Verified	*Only the named recipient is credited the YES legs.*	Report
convertConservesSupply	Verified	*Total YES supply rises by the amount per other leg while the source NO supply falls by the amount.*	Report

NEGRISK-HORIZONTAL-01

Status: Verified	horizontal split and merge cover exactly the YES legs of an event with equal amounts, and are exact inverses.

Rule Name	Status	Description	Link to rule report
horizontalSplitMintsEachYesExactly	Verified	*Every covered leg of the event, including the synthetic Other, is credited exactly the split amount to the recipient.*	Report
horizontalSplitBurnsCollateralExactly	Verified	*A horizontal split burns exactly the requested amount of pUSD from the module's own balance.*	Report
horizontalSplitBurnsOnlyModuleCollateral	Verified	*A horizontal split changes no collateral balance other than the module's.*	Report
horizontalSplitDoesNotMintNo	Verified	*A horizontal split never mints a NO position; it touches YES legs only.*	Report

horizontalSplitDoesNotMintBeyondOther	Verified	*Coverage stops at the synthetic Other condition; the YES leg one past it is untouched.*	Report
horizontalSplitCreditsOnlyRecipient	Verified	*Only the named recipient is credited; any other account's covered YES balance is untouched.*	Report
horizontalSplitTouchesOnlyCoveredYes	Verified	*Any position id outside the covered YES legs is untouched for every account.*	Report
horizontalMergeBurnsEachYesExactly	Verified	*Every covered leg of the event is burned by exactly the merge amount from the module.*	Report
horizontalMergeMintsCollateralExactly	Verified	*A horizontal merge mints exactly the requested amount of pUSD to the recipient.*	Report
horizontalMergeMintsOnlyRecipientCollateral	Verified	*A horizontal merge changes no collateral balance other than the recipient's.*	Report
horizontalMergeDoesNotBurnNo	Verified	*A horizontal merge never burns a NO position; it touches YES legs only.*	Report
horizontalMergeDoesNotBurnBeyondOther	Verified	*A horizontal merge does not burn the YES leg past the synthetic Other condition.*	Report
horizontalMergeBurnsOnlyModuleYes	Verified	*A horizontal merge burns the covered legs from the module only; any other account's covered YES leg is untouched.*	Report

horizontalMergeTouchesOnlyCoveredYes	Verified	*Any position id outside the covered YES legs is untouched for every account.*	Report
horizontalSplitMergeInverse	Verified	*A horizontal split followed by a horizontal merge restores the position supplies and the collateral balances.*	Report

NEGRISK-MERGE-01

Status: Verified	NegRiskModule merge is the exact inverse of split.

Rule Name	Status	Description	Link to rule report
mergeBurnsYesAndNoExactly	Verified	*A successful merge burns exactly the requested amount of the YES and NO positions from the module.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
mergeMintsCollateralExactly	Verified	*A successful merge mints exactly the requested amount of pUSD to the recipient.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
mergeMintsOnlyRecipientCollateral	Verified	*A successful merge changes no collateral balance other than the recipient's.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
mergeBurnsOnlyModulePositions	Verified	*A successful merge touches no position balance other than the two legs held by the module.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

NEGRISK-REDEEM-01

Status: Verified	NegRiskModule redeem pays the floor of the resolved payout, bounded by the amount, and reverts on an unresolved condition or an invalid outcome.

Rule Name	Status	Description	Link to rule report
resultNormalized	Verified	*Every stored result is a complementary pair summing to the result denominator.*	Report
redeemPaysExactFloor	Verified	*A successful redeem mints exactly the floor of the amount scaled by the resolved numerator.*	Report
redeemPayoutNeverExceedsAmount	Verified	*For a resolved condition the payout never exceeds the redeemed amount.*	Report
redeemBurnsPositionExactly	Verified	*A successful redeem burns exactly the redeemed amount from the module and lowers supply by the same.*	Report
redeemBurnsOnlyThatPosition	Verified	*A successful redeem touches no position balance other than the redeemed one.*	Report
redeemMintsOnlyRecipientCollateral	Verified	*A successful redeem changes no collateral balance other than the recipient's.*	Report
redeemRevertsWhenUnresolved	Verified	*redeem reverts when the condition has no stored result.*	Report
redeemRevertsOnBadOutcome	Verified	*redeem reverts when the outcome index is out of range.*	Report

NEGRISK-SPLIT-01

Status: Verified	NegRiskModule split mints equal YES and NO and burns exactly the pre-transferred collateral.

Rule Name	Status	Description	Link to rule report
splitMintsYesAndNoExactly	Verified	*A successful split credits each recipient by exactly the requested amount of the YES and NO positions.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
splitBurnsCollateralExactly	Verified	*A successful split burns exactly the requested amount of pUSD from the module's own balance.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
splitBurnsOnlyModuleCollateral	Verified	*A successful split changes no collateral balance other than the module's.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
splitTouchesOnlySplitPositions	Verified	*A successful split touches no position balance other than the two it credits.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

NegRiskModule Resolution Integrity

Module General Assumptions

- Loops are unrolled to between 3 and 5 iterations, stated per rule where it is not the upper bound.
- The legacy condition-id helpers are summarized in CVL.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.

Module Properties

MODU-INT-01

Status: Verified	the per-event result counter always equals the sum of the YES numerators of the conditions resolved into that event.

Rule Name	Status	Description	Link to rule report
resultsSumTracksConditions	Verified	*For every event the running result counter equals the sum of the YES numerators of its resolved conditions.*	Report
mirrorNormalized	Verified	*A resolved binary condition has payout numerators summing to the denominator.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
lengthMatchesReal	Verified	*The mirrored result length stays in sync with the real result mapping.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
resultLengthNormalized	Verified	*A stored result is either absent or a full complementary pair; no other length is reachable.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
valueMatchesReal	Verified	*The mirror's values equal the real array's values, not just its length.*	Report

		Assumption: Verified with loops unrolled to at most 3 iterations.	
resultsAreBinary	Verified	*Every neg-risk condition resolves fully YES or fully NO.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
resultsSumIsFlag	Verified	*The per-event result sum only ever holds zero or the denominator, so it behaves as a resolution flag.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
conditionsResolvedCount	Verified	*The contract's resolved-condition counter matches the number of conditions with a stored result.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
resultsSumIsResolvedYesSum	Verified	*The contract's result-sum counter matches the sum of stored YES numerators across resolved conditions.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

NEGRISK-PARTITION-01

Status: Verified	The YES numerators of all its conditions plus the synthetic Other sum to exactly the denominator once resolution completes.

Rule Name	Status	Description	Link to rule report
resultsSumBounded	Verified	*The sum of resolved YES numerators of an event never exceeds the denominator.*	Report
resultsSumTracksYesNumerators	Verified	*The tracked event sum always equals the sum of the YES numerators actually stored on its conditions.*	Report

resolvedResultNormalized	Verified	*Every resolved condition of the event carries a complementary pair summing to the denominator.*	Report
otherResolvedCompletesPartition	Verified	*Resolving the synthetic Other condition brings the event sum to exactly the denominator.*	Report
fullResolutionImpliesFullSum	Verified	*Once every condition of an event is resolved, the event sum is exactly the denominator.*	Report

NEGRISK-PARTITION-02

Status: Verified	The resolved-condition counter of an event is monotone and never exceeds the number of conditions it has.

Rule Name	Status	Description	Link to rule report
conditionsResolvedMonotone	Verified	*No method decreases the resolved-condition counter of an event.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
resolvedConditionImmutable	Verified	*Each condition resolves at most once and never returns to unresolved.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
counterMatchesResolvedCount	Verified	*The resolved-condition counter equals the number of conditions of the event that carry a stored result.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
conditionsResolvedBounded	Verified	*The resolved-condition counter never exceeds the number of conditions of the event.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

NEGRISK-RESULT-NORMALIZED-01

Status: Verified	Every NegRiskModule stored result is a complementary pair summing to the result denominator, and reportResult rejects a malformed vector.

Rule Name	Status	Description	Link to rule report
resultNormalized	Verified	*For every condition with a stored result, the result is a pair summing to the result denominator.* *Assumption: The migration step is bounded to at most 2 migrated positions per call.*	Report
reportResultRevertsOnMalformed	Verified	*reportResult reverts on any result vector that is not a complementary pair.*	Report
reportResultCannotChangeStoredResult	Verified	*reportResult can never overwrite a result that is already stored.*	Report
payoutConservation	Verified	*The YES and NO payouts of a resolved condition together account for the amount that funded it.*	Report
migrationStoredResultIsBinary	Verified	*A result stored by the migration path is always a definitive binary outcome.*	Report

NegRiskModule Bridge Authorization

Module General Assumptions

- Loops are unrolled to at most 5 iterations.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.

Module Properties

NEGRISK-BRIDGE-02			
Status: Verified		NegRiskModule mintFromBridge and burnFromBridge are bridge-role-only and move exactly the tuple they are given.	
Rule Name	Status	Description	Link to rule report
mintFromBridge OnlyBridge	Verified	*Only an address holding the bridge role can mint; without it the call reverts.*	Report
mintFromBridge CreditsRecipient	Verified	*A successful bridge mint credits exactly the requested amount to the recipient and raises supply by the same.*	Report
mintFromBridge NoOtherBalance Change	Verified	*A bridge mint touches no balance other than the credited one.*	Report
burnFromBridge OnlyBridge	Verified	*Only an address holding the bridge role can burn; without it the call reverts.*	Report
burnFromBridge DebitsModule	Verified	*A successful bridge burn debits the module by exactly the total targeting each id and lowers supply by the same.*	Report
burnFromBridge NoOtherAddress Change	Verified	*A bridge burn only ever debits the module; no other holder is touched.*	Report

CombinatorialModule Global Solvency

Module General Assumptions

- Loops are unrolled to between 2 and 3 iterations, stated per rule where it is not the upper bound.
- The legacy condition-id helpers are summarized in CVL.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.
- The combinatorial payout is summarized with an equivalent CVL model.

Module Properties

COMBO-GLOB-SOLVENCY

Status: Verified	CombinatorialModule preserves the collateral backing of every outstanding position it has minted, under every possible market resolution.

Rule Name	Status	Description	Link to rule report
solvencyPreserved	Verified	*split, merge and redeem on a prepared combinatorial conjunction preserve the backing inequality against the result-aware worst-case liability.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report

COMBO-MODULE-ESCROW-01

Status: Verified	No CombinatorialModule function increases the module's own pUSD balance, or its balance of any position id, beyond what the caller explicitly directed to it.

Rule Name	Status	Description	Link to rule report
moduleNeverAccumulatesCollateral	Verified	*The module's own pUSD balance grows by at most the amount the caller explicitly directed to it.*	Report
moduleNeverAccumulatesPositions	Verified	*The module's own balance of every position id grows by at most the amount the caller explicitly minted to it for that id.*	Report Report

COMBO-NEUTRAL-01

Status: Verified	The position-only combinatorial transforms move no collateral.

Rule Name	Status	Description	Link to rule report
transformsAreCollateralNeutral	Verified	*None of the eleven position-only combinatorial transforms changes the pUSD supply or any counted asset balance.*	Report

CombinatorialModule Position Lifecycle

Module General Assumptions

- Loops are unrolled to between 2 and 3 iterations, stated per rule where it is not the upper bound.
- The legacy condition-id helpers are summarized in CVL.
- The combinatorial payout is summarized with an equivalent CVL model.

Module Properties

COMBO-SPLIT-MERGE-01			
Status: Verified		Combinatorial split and merge are exact inverses and conserve collateral against the YES and NO pair.	
Rule Name	Status	Description	Link to rule report
splitEffects	Verified	*A successful split raises the YES and NO supplies by the amount, credits the two recipients, and burns exactly that much collateral.*	Report
splitOnlyCombinatorial	Verified	*split rejects any condition that is not a combinatorial one.*	Report
mergeEffects	Verified	*A successful merge lowers the YES and NO supplies by the amount, debits the module's pre-transferred legs, and mints exactly that much collateral.*	Report
splitThenMergeIsInverse	Verified	*split followed by merge restores the position supplies, the module leg balances and the collateral supply to their pre-split values.*	Report
mergeThenSplitIsInverse	Verified	*merge followed by split re-mints the pair and re-burns the collateral, restoring the pre-merge state.*	Report

certora

COMBO-WRAP-01

Status: Verified	Wrap and unwrap are a value-preserving bijection between an underlying position and its single-leg combinatorial form.

Rule Name	Status	Description	Link to rule report
wrapValueConserving	Verified	*wrap burns the underlying position and mints exactly the single-leg combinatorial YES, and the two redeem for exactly equal value.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report
unwrapValueConserving	Verified	*unwrap burns the single-leg combinatorial position and mints exactly the matching underlying, and the two redeem for exactly equal value.* *Assumption: Verified with loops unrolled to at most 2 iterations.*	Report

CombinatorialModule Refinement Value Conservation

Module General Assumptions

- Loops are unrolled to at most 2 iterations.
- The combinatorial payout is summarized with an equivalent CVL model.
- The legacy condition-id helpers are summarized in CVL.

Module Properties

COMBO-COMPRESS-01			
Status: Verified		Compress pays out collateral and a residual worth no more than the position it consumed.	
Rule Name	Status	Description	Link to rule report
compressResidualValueConserving	Verified	*When one leg is resolved with a non-zero factor and one is unresolved, the collateral paid plus the residual redeem for no more than the input at every complete resolution.* *Assumption: Case split covering one leg resolved with a non-zero payout factor and one leg unresolved.*	Report
compressTerminalZeroValueConserving	Verified	*When a resolved leg pays zero no residual is minted and the collateral paid does not exceed the input value.* *Assumption: Case split covering one leg resolved with a zero payout factor and one leg unresolved.*	Report
compressAllResolvedValueConserving	Verified	*When every leg is resolved compress mints no residual and pays out collateral worth no more than the input.* *Assumption: Case split covering every leg being resolved.*	Report

COMBO-REFINE-FORWARD-01

Status: Verified	The forward refinements mint exactly the expected fine-grained positions and never create redemption value.

Rule Name	Status	Description	Link to rule report
splitOnConditionValueConserving	Verified	*splitOnCondition mints exactly the two children and burns the parent, and the children redeem for no more than the parent at every complete resolution.*	Report
extractValueConserving	Verified	*extract mints exactly the reduced and residual positions and burns the full one, and the outputs redeem for no more than the input at every complete resolution.*	Report
convertToYesBasketValueConserving	Verified	*convertToYesBasket mints exactly the basket positions and burns the full one, and the basket redeems for no more than the input at every complete resolution.*	Report

COMBO-REFINE-INVERSE-01

Status: Verified	The inverse refinements mint the coarse position for no more than the parts were worth, up to one wei of recovered rounding dust.

Rule Name	Status	Description	Link to rule report
mergeOnConditionValueConserving	Verified	*mergeOnCondition mints the parent and burns the two children, and the parent redeems for no more than the children plus one wei.*	Report
injectValueConserving	Verified	*inject mints the full position and burns the reduced and residual parts, and the full position redeems for no more than the parts plus one wei.*	Report

mergeFromYesBasketValueConserving	Verified	*mergeFromYesBasket mints the full position and burns the basket, and the full position redeems for no more than the basket plus one wei.*	Report

COMBO-ROUNDTRIP-01

Status: Verified	A forward refinement followed by its inverse is exactly value-neutral, with no rounding dust.

Rule Name	Status	Description	Link to rule report
splitThenMergeOnConditionIsValueNeutral	Verified	*splitOnCondition followed by mergeOnCondition returns exactly the original position, minting and burning equal value.*	Report
extractThenInjectIsValueNeutral	Verified	*extract followed by inject returns exactly the original position, minting and burning equal value.*	Report
convertThenMergeFromYesBasketIsValueNeutral	Verified	*convertToYesBasket followed by mergeFromYesBasket returns exactly the original position, minting and burning equal value.*	Report

CombinatorialModule Conjunction Store

Module General Assumptions

- Loops are unrolled to at most 3 iterations.
- The legacy condition-id helpers are summarized in CVL.
- The combinatorial payout is summarized with an equivalent CVL model.

Module Properties

COMBI-SL-FIX-1

Status: Verified	A stored leg definition is authoritative: an occupied slot rejects a different definition.

Rule Name	Status	Description	Link to rule report
storeLegsRevertsOnDefinitionMismatch	Verified	*Presenting a different leg definition for an occupied slot reverts.*	Report
storeLegsSucceedsOnExactMatch	Verified	*Presenting the identical leg definition for an occupied slot succeeds.*	Report
storeLegsWritesInputWhenFresh	Verified	*A fresh slot stores exactly the presented leg definition.*	Report

COMBI-SL-FIX-2

Status: Verified	The internal leg-store helper is the only writer of the leg store.

Rule Name	Status	Description	Link to rule report
legsChangeOnlyThroughStoreLegs	Verified	*No entry point changes the leg store except through the internal store helper.* *Assumption: The ownership-handover entry points are excluded because they never touch the leg store and their two-step flow needs a time warp to be meaningful.*	Report

COMBI-SL-FIX-3

Status: Verified	Once stored, a leg definition never changes.

Rule Name	Status	Description	Link to rule report
storedLegCountImmutable	Verified	*Once a definition is stored, its length never changes again.* *Assumption: The ownership-handover entry points are excluded because they never touch the leg store and their two-step flow needs a time warp to be meaningful.*	Report
storedLegsImmutable	Verified	*Once a definition is stored, no element of it ever changes.* *Assumption: The ownership-handover entry points are excluded because they never touch the leg store and their two-step flow needs a time warp to be meaningful.*	Report

COMBO-PREPARE-COMBO-01

Status: Verified	CombinatorialModule prepareCondition binds a condition id to its leg array, and a prepared condition never becomes unprepared.

Rule Name	Status	Description	Link to rule report
prepareConditionReturnsIdentifierOfLegs	Verified	*prepareCondition returns the condition id derived from exactly the leg array it was given.*	Report
prepareConditionPreparesTheCondition	Verified	*After a successful prepareCondition the returned condition is prepared.*	Report
prepareConditionFreshStoresExactLegs	Verified	*Preparing a fresh condition stores exactly the supplied leg array.*	Report
prepareConditionOccupiedLegsUnchanged	Verified	*Re-preparing an already-prepared condition leaves its stored legs unchanged.*	Report
prepareConditionOnlyCanonicalLegs	Verified	*prepareCondition rejects any leg array that is not canonical.*	Report
preparedConditionStaysPrepared	Verified	*No entry point can make a prepared condition unprepared.*	Report

COMBO-STORE-01

Status: Verified	Every stored combinatorial conjunction is well-formed and canonical.

Rule Name	Status	Description	Link to rule report
storedConjunctionsCanonical	Verified	*Every stored conjunction is non-empty, sits on binary or neg-risk legs with a valid outcome, and is strictly ascending with distinct condition ids.*	Report

storedConjunctionsWellFormed	Verified	*Every stored conjunction hashes back to its own condition id.*	Report

PositionManager Token Integrity

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

PM-INT-RULE-01			
Status: Verified		mint, batchMint, burn and batchBurn move exactly the requested amounts and revert exactly on their documented causes.	
Rule Name	Status	Description	Link to rule report
mintIntegrity	Verified	*mint credits the recipient with exactly the requested amount and changes nothing else.*	Report
mintReverts	Verified	*mint reverts exactly when the caller is unauthorized, the recipient is zero, the balance would overflow, or value was sent.*	Report
batchMintIntegrity	Verified	*batchMint credits the recipient with exactly the per-id sum of the amounts and changes nothing else.*	Report
batchMintReverts	Verified	*batchMint reverts exactly when the caller is unauthorized, the lengths mismatch, the recipient is zero, any id would overflow, or value was sent.*	Report
burnIntegrity	Verified	*burn debits the caller by exactly the requested amount and changes nothing else.*	Report
burnReverts	Verified	*burn reverts exactly when the caller is unauthorized, the balance is insufficient, or value was sent.*	Report
batchBurnIntegrity	Verified	*batchBurn debits the caller by exactly the per-id sum of the amounts and changes nothing else.*	Report

batchBurnReverts	Verified	*batchBurn reverts exactly when the caller is unauthorized, the lengths mismatch, any id has an insufficient balance, or value was sent.*	Report

PositionManager Solvency

Module General Assumptions

- Loops are unrolled to at most 3 iterations.
- The legacy condition-id helpers are summarized in CVL.
- PositionManager mint, burn and transfer are replaced by an equivalent CVL model.

Module Properties

PM-GLOB-SOLVENCY

Status: Verified	PositionManager preserves the collateral backing of every outstanding position it has minted, under every possible market resolution.

Rule Name	Status	Description	Link to rule report
solvencyPreserved	Verified	*Every PositionManager method preserves the backing inequality between counted assets and the sum of pUSD supply and worst-case liability.*	Report

PM-SUPPLY-01

Status: Verified	Position supply changes only through the mint and burn entry points.

Rule Name	Status	Description	Link to rule report
positionSupplyConservation	Verified	*Position supply per id changes only through the mint and burn entry points.*	Report
unsafeTransferIsExact	Verified	*unsafeTransferFrom debits the sender and credits the recipient by exactly the requested amount, and moves no other balance cell.*	Report

unsafeBatchTransferConservesThePair	Verified	*unsafeBatchTransferFrom leaves the sender-plus-recipient total unchanged for every id, and moves no third party.*	Report
safeTransferIsExact	Verified	*safeTransferFrom debits the sender and credits the recipient by exactly the requested amount, and moves no other balance.*	Report
safeBatchTransferConservesThePair	Verified	*safeBatchTransferFrom leaves the sender-plus-recipient total unchanged for every id, and moves no third party.*	Report

PositionManager Access Control

Module General Assumptions

- Loops are unrolled to at most 3 iterations.
- Solady OwnableRoles role bitmaps are replaced by a CVL model.

Module Properties

ACCESS-PM-MINT-01

Status: Verified	Only the registered module for a positionId, or a crossModuleAuth module, can mint or burn that position.

Rule Name	Status	Description	Link to rule report
mintRequiresModuleAuth	Verified	*mint succeeds only for the id's registered module or a cross-authorized caller.*	Report
burnRequiresModuleAuth	Verified	*burn succeeds only for the id's registered module or a cross-authorized caller.*	Report
batchMintRequiresModuleAuth	Verified	*batchMint succeeds only when the caller is the registered module for every id, or is cross-authorized.*	Report
batchBurnRequiresModuleAuth	Verified	*batchBurn succeeds only when the caller is the registered module for every id, or is cross-authorized.*	Report
balanceChangesRequireAuthorizedEntryPoint	Verified	*A balance of (holder, id) changes only via a transfer or via an authorized mint or burn entry point.*	Report

ACCESS-PM-REGISTRY-01

Status: Verified	Module registration, removal, and crossModuleAuth are admin-only.

Rule Name	Status	Description	Link to rule report
addModuleRequiresAdmin	Verified	*addModule succeeds only for an admin caller.*	Report
removeModuleRequiresAdmin	Verified	*removeModule succeeds only for an admin caller.*	Report
setCrossModuleAuthRequiresAdmin	Verified	*setCrossModuleAuth succeeds only for an admin caller.*	Report
registryChangesRequireAdmin	Verified	*moduleById and crossModuleAuth change only via the three registry entry points, and only under an admin caller.*	Report

ACCESS-URI-PAYOUT-01

Status: Verified	uri and getPayout revert for a position whose module is not registered, instead of calling into the zero address.

Rule Name	Status	Description	Link to rule report
uriRevertsWhenUnregistered	Verified	*uri reverts when the position's module is not registered.*	Report
uriSucceedsWhenRegistered	Verified	*For a registered module uri does not revert; the registration guard is its only revert cause.*	Report
getPayoutRevertsWhenUnregistered	Verified	*getPayout reverts when the position's module is not registered, before ever calling the module.*	Report

getPayoutSucceedsWhenRegistered	Verified	*For a registered module getPayout forwards to it exactly once with the arguments unchanged and returns its result verbatim.*	Report

PM-ACCESS-INV-01

Status: Verified	a module holds cross-module mint and burn authorization only while it is registered under its own id.

Rule Name	Status	Description	Link to rule report
crossAuthImpliesRegistered	Verified	*A cross-authorized module is always registered under its own reported id.*	Report
moduleReportsOwnId	Verified	*A registered module always reports the id it is filed under.*	Report
setCrossAuthRequiresRegistration	Verified	*Cross-module authorization can only be granted to a module registered under its own id.*	Report
removeModuleClearsCrossAuth	Verified	*De-registering a module clears its cross-module authorization.*	Report
crossAuthOnlyByAdminEntryPoints	Verified	*A cross-module authorization flag changes only through the two admin-gated entry points.*	Report

Exchange Collateral Conservation

Module General Assumptions

- Loops are unrolled to between 3 and 5 iterations, stated per rule where it is not the upper bound.

Module Properties

EXCHANGE-COLLATERAL-CONSERVATION-01			
Status: Verified		matchOrders is a pass-through router: a successful match neither mints nor drains collateral, and the Exchange residual is exactly the fees it retains.	
Rule Name	Status	Description	Link to rule report
residualBatchSell	Verified	*The Exchange's pUSD balance is identical before and after a batch sell match.*	Report
residualBatchBuy	Verified	*The Exchange's pUSD balance is identical before and after a batch buy match.*	Report
residualComplementarySell	Verified	*On the complementary sell executor the Exchange receives fill plus fee from each maker and pays the taker its net proceeds, retaining exactly the fees.*	Report
residualComplementaryBuy	Verified	*On the complementary buy path the Exchange takes no custody of pUSD, so its own balance is unchanged.*	Report
residualComplementarySellFull	Verified	*Over the full complementary sell wrapper the Exchange forwards the parked fees onward, so its net balance change is zero.*	Report
feeBatchSell	Verified	*A batch sell match credits the fee receiver exactly the taker fee plus the sum of maker fees.*	Report
feeBatchBuy	Verified	*A batch buy match credits the fee receiver exactly the taker fee plus the sum of maker fees.*	Report

feeComplementaryBuy	Verified	*A complementary buy routes collateral peer to peer and sends fees to the fee receiver from the taker, never through the Exchange.*	Report
feeComplementarySell	Verified	*A complementary sell forwards exactly the taker fee plus the sum of maker fees to the fee receiver.*	Report
takerCreditComplementaryZeroFeeSell	Verified	*A zero-fee complementary sell routes collateral peer to peer and credits the taker exactly the sum of maker fills.*	Report
takerCreditedFillsMinusFee_len1	Verified	*With 1 maker fill, the taker is credited exactly the accumulated fills minus its own fee.* *Assumption: Case split covering a fixed maker count, so the loop is fully unrolled.*	Report
takerCreditedFillsMinusFee_len2	Verified	*With 2 maker fills, the taker is credited exactly the accumulated fills minus its own fee.* *Assumption: Case split covering a fixed maker count, so the loop is fully unrolled.*	Report
takerCreditedFillsMinusFee_len3	Verified	*With 3 maker fills, the taker is credited exactly the accumulated fills minus its own fee.* *Assumption: Case split covering a fixed maker count, so the loop is fully unrolled.*	Report
takerCreditedFillsMinusFee_len4	Verified	*With 4 maker fills, the taker is credited exactly the accumulated fills minus its own fee.* *Assumption: Case split covering a fixed maker count, so the loop is fully unrolled.*	Report
takerCreditedFillsMinusFee_len5	Verified	*With 5 maker fills, the taker is credited exactly the accumulated fills minus its own fee.* *Assumption: Case split covering a fixed maker count, so the loop is fully unrolled.*	Report
takerDebitBatchBuy	Verified	*On a batch buy the taker's own balance falls by exactly its making amount plus fee, with the remainder refunded.*	Report

takerCreditBatchSell	Verified	*On a batch sell the taker's own balance rises by exactly its taking amount net of fee.*	Report
matchOrdersNoMintNoDrain	Verified	*Exchange is neither drained nor left holding more than the declared fees.* *Assumption: Case split covering a single maker, with the dispatch, the executors and every collateral transfer running for real.*	Report
matchOrdersCombinatorialNoMintNoDrain	Verified	*Exchange is neither drained nor left holding more than the declared fees.* *Assumption: Case split covering a single maker, with the dispatch, the executors and every collateral transfer running for real.*	Report

EXCHANGE-SELL-ACCUMULATOR-01

Status: Verified	The complementary-sell accumulator equals the sum of the maker fills.

Rule Name	Status	Description	Link to rule report
sellAccumulatorEqualsFills_len1	Verified	*With 1 maker fill, the executor's returned taking amount equals the sum of the fills.* *Assumption: Case split covering a fixed maker count, so the accumulator is checked without a loop.*	Report
sellAccumulatorEqualsFills_len2	Verified	*With 2 maker fills, the executor's returned taking amount equals the sum of the fills.* *Assumption: Case split covering a fixed maker count, so the accumulator is checked without a loop.*	Report
sellAccumulatorEqualsFills_len3	Verified	*With 3 maker fills, the executor's returned taking amount equals the sum of the fills.* *Assumption: Case split covering a fixed maker count, so the accumulator is checked without a loop.*	Report

sellAccumulatorEqualsFills_len4	Verified	*With 4 maker fills, the executor's returned taking amount equals the sum of the fills.* *Assumption: Case split covering a fixed maker count, so the accumulator is checked without a loop.*	Report
sellAccumulatorEqualsFills_len5	Verified	*With 5 maker fills, the executor's returned taking amount equals the sum of the fills.* *Assumption: Case split covering a fixed maker count, so the accumulator is checked without a loop.*	Report

EXCHANGE-TAKER-AMOUNT-01

Status: Verified	The taker receives exactly the operator-declared receive amount on every path.

Rule Name	Status	Description	Link to rule report
takerBuyComplementary	Verified	*On the complementary buy path the taker receives exactly the sum of the fills in position tokens.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
takerBuyBatch	Verified	*On a batch buy the taker receives exactly the declared receive amount in position tokens.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
takerSellComplementaryZeroFee	Verified	*On the zero-fee complementary sell path the taker receives exactly the sum of the fills in collateral.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
takerSellComplementaryWithFee	Verified	*On the fee-carrying complementary sell path the taker receives exactly its taking amount net of the taker fee.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

takerSellBatch	Verified	*On a batch sell the taker receives exactly the declared receive amount net of the taker fee.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
takerSellPublicEntries	Verified	*Driving the real matching entry points end to end, the taker receives exactly the declared receive amount net of the taker fee.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
takerBuyComplementaryBinding	Verified	*At the complementary matching step the taker's credit is bound to the declared receive amount.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

FEE-CAP-01b

Status: Verified	A per-fill fee never exceeds the maximum rate applied to the cash value, nor the collateral proceeds of the party paying it.

Rule Name	Status	Description	Link to rule report
validateFeeExactness	Verified	*The public fee checker reverts exactly when a non-zero fee exceeds the enabled cap or the cap computation overflows.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
makerFeeCappedByRate	Verified	*Every maker fill's fee is bounded by the maximum rate applied to its cash value, on both public entry points.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
sellMakerFeeSmallerOrEqualThanProceeds	Verified	*A selling maker's fee never exceeds the collateral proceeds of that fill, even at a zero rate.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

sellTakerFeeBounds	Verified	*A selling taker's fee is bounded both by the maximum rate applied to its cash value and by its proceeds.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report
buyTakerFeeCappedByRate_batch	Verified	*A buying taker's fee is bounded by the maximum rate applied to the batch cash value derived from the refund accounting.* *Assumption: Verified with loops unrolled to at most 3 iterations.*	Report

Exchange Order Authorization

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

EXCHANGE-SIG-01a	
Status: Verified	An order fills only if it was preapproved or the verifier accepted a signature over the order own hash.

Rule Name	Status	Description	Link to rule report
fillRequiresConsent	Verified	*Any change to any order's fill status requires either preapproval or an accepted signature.*	Report
takerFillRequiresConsent	Verified	*Consent is required on every successful taker fill.*	Report
makerFillRequiresConsent	Verified	*Consent is required on every successful maker fill, for an arbitrary maker index.*	Report
preapprovalOnlyViaSignedPreapprove	Verified	*A preapproval flag turns true only inside the preapprove entry point, and only after the verifier accepted the hash.*	Report

EXCHANGE-SIG-01b	
Status: Verified	The signature verifier accepts only signatures that bind the maker, across all four signature types.

Rule Name	Status	Description	Link to rule report
eoaSignatureBindsMaker	Verified	*Acceptance requires the signer to be the maker and a well-formed ECDSA signature recovering to that signer.*	Report
erc1271SignatureBindsMaker	Verified	*Acceptance requires the signer to be the maker and the maker wallet's approval of exactly this hash.*	Report
proxySignatureBindsMaker	Verified	*Acceptance requires the maker to be the signer's derived proxy wallet, plus a well-formed signature by the signer.*	Report
safeSignatureBindsMaker	Verified	*Acceptance requires the maker to be the safe wallet derived from the signer, plus a well-formed signature by the signer.*	Report
emptySigPassesIffPreapproved	Verified	*With an empty signature the consent gate passes exactly when the hash is preapproved.*	Report
signedPassesIffVerifierAccepts	Verified	*With a non-empty signature the gate passes exactly when the verifier accepts, and the preapproval flag is never consulted.*	Report

EXCHANGE-USER-PAUSE-01

Status: Verified	User pause is self-service, block-delayed, and blocks the user orders once effective.

Rule Name	Status	Description	Link to rule report
pauseUserExactness	Verified	*pauseUser reverts when a pause is already scheduled and otherwise schedules exactly the current block plus the interval, touching nobody else.*	Report
unpauseUserExactness	Verified	*unpauseUser always succeeds and clears exactly the caller's own pause slot.*	Report

isUserPausedDefinition	Verified	*In every state the paused predicate is exactly that the slot is set and its block has been reached.*	Report
pauseUserDelayWindow	Verified	*After pauseUser the user is not paused at any block before the scheduled one and paused at every block from it onward.*	Report
pauseUserIntervalZeroIsImmediate	Verified	*With a zero interval the pause takes effect in the calling block.*	Report
pausedMakerOrderCannotBeFilled	Verified	*Any match including a maker order whose maker is paused at call time reverts.*	Report
pausedTakerOrderCannotBeFilled	Verified	*Any match whose taker is paused at call time reverts.*	Report
userPauseAuthorization	Verified	*A pause slot is set only by its own user through pauseUser, cleared only through unpauseUser, and never modified in place.*	Report
setUserPauseBlockIntervalExactness	Verified	*The pause interval can only be set by an admin and only to a value within the cap.*	Report
intervalCapped	Verified	*The configured pause interval never exceeds its cap in any reachable state.*	Report

Exchange Order Accounting

Module General Assumptions

- Loops are unrolled to at most 5 iterations.

Module Properties

ORDER-FILL-MONOTONE-01			
Status: Verified		Order fill is monotone and bounded: remaining never rises, cumulative fills never exceed the order amount, and the filled flag is never cleared.	
Rule Name	Status	Description	Link to rule report
storePackingFaithful	Verified	*The packed order status encodes the filled flag as remaining == 0 and preserves the remaining bits.*	Report
fillMonotoneDecreasing_makerPath	Verified	*On the maker path one fill lowers remaining by exactly the fill amount and never below zero, so the consumed amount is bounded by the order amount.*	Report
fillMonotoneDecreasing_takerPath	Verified	*The complementary taker path decreases remaining by exactly the fill amount through the other store caller.*	Report
cumulativeFillsBounded	Verified	*Two sequential maker fills consume no more than the starting remaining amount.*	Report
overfillReverts	Verified	*Attempting to fill more than the supplied order amount always reverts.*	Report
filledLatchNeverCleared	Verified	*No in-scope method clears a set filled flag, so no entry point can re-open a filled order.*	Report
filledLatchIrreversible_makerPath	Verified	*Once filled, the maker-path validation reverts, so no further store can run and the latch can never be cleared.*	Report

filledLatchIrreversible_publicEntry	Verified	*A filled taker order cannot be matched again through the standard public entry point.*	Report
filledLatchIrreversible_publicEntryCombinatorial	Verified	*A filled taker order cannot be matched again through the combinatorial public entry point either.*	Report

Exchange Combinatorial Binding

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

EXCHANGE-PREPARE-COMBO-01			
Status: Verified		The Exchange combinatorial entry point binds the condition it prepares atomically to the taker token id.	
Rule Name	Status	Description	Link to rule report
bindPrefixTakerTokenBoundToPreparedLegs	Verified	*The condition prepared atomically before matching is exactly the one encoded in the taker token id.*	Report
bindPrefixTakerTokenIsCombinatorial	Verified	*The combinatorial entry point only accepts a taker token id on the combinatorial module.*	Report
bindPrefixTakerConditionPreparedAfter	Verified	*After the combinatorial entry point succeeds, the taker's condition is prepared.*	Report
entryTakerOutcomeIsYesOrNo	Verified	*The combinatorial entry point only accepts a taker token whose outcome index is a valid YES or NO leg.*	Report

CollateralToken Wrap Backing

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

UNWRAP-01			
Status: Verified		unwrap burns pUSD exactly matched by the asset it releases from the vault.	
Rule Name	Status	Description	Link to rule report
unwrapReleasesAndBurns	Verified	*A successful unwrap releases exactly the requested asset from the vault to the recipient and burns the same amount of pUSD from the ramp's own balance.*	Report
unwrapReleasesAndBurnsLegacyOverload	Verified	*A successful unwrap through the five-argument overload releases exactly the requested asset from the vault to the recipient and burns the same amount of pUSD from the ramp's own balance, whatever the callback arguments.*	Report
unwrapTouchesNoOtherBalance	Verified	*unwrap touches no pUSD balance other than the ramp's and no asset cell other than the vault and the recipient.*	Report
unwrapTouchesNoOtherBalanceLegacyOverload	Verified	*unwrap through the five-argument overload touches no pUSD balance other than the ramp's and no asset cell other than the vault and the recipient, whatever the callback arguments.*	Report

WRAP-01		
Status: Verified		wrap mints pUSD exactly backed by the asset it forwards to the vault.

Rule Name	Status	Description	Link to rule report
wrapMintsAndForwards	Verified	*A successful wrap mints exactly the requested pUSD to the recipient and forwards the same amount of the asset to the vault.*	Report
wrapMintsAndForwardsLegacyOverload	Verified	*A successful wrap through the five-argument overload mints exactly the requested pUSD to the recipient and forwards the same amount of the asset to the vault, whatever the callback arguments.*	Report
wrapTouchesNoOtherBalance	Verified	*wrap touches no pUSD balance other than the recipient and no asset cell other than the ramp and the vault.*	Report
wrapTouchesNoOtherBalanceLegacyOverload	Verified	*wrap through the five-argument overload touches no pUSD balance other than the recipient and no asset cell other than the ramp and the vault, whatever the callback arguments.*	Report

CollateralToken Supply Access Control

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

ACCESS-COLLATERAL-MINT-01			
Status: Verified		Only the minter role can change pUSD supply through mint and burn, and only the wrapper role on a valid asset through wrap and unwrap.	
Rule Name	Status	Description	Link to rule report
onlySupplyMutators	Verified	*No method other than mint, burn, wrap and unwrap changes the pUSD total supply.*	Report
supplyChangeRequiresRole	Verified	*Any change to pUSD total supply requires the caller to hold the minter or wrapper role.*	Report
mintRevertsWithoutMinterRole	Verified	*mint reverts for a caller without the minter role.*	Report
burnRevertsWithoutMinterRole	Verified	*burn reverts for a caller without the minter role.*	Report
wrapRevertsWithoutWrapperRoleOrInvalidAsset	Verified	*wrap reverts for a caller without the wrapper role or for an asset that is not accepted.*	Report
unwrapRevertsWithoutWrapperRoleOrInvalidAsset	Verified	*unwrap reverts for a caller without the wrapper role or for an asset that is not accepted.*	Report

upgradeRevertsWithoutOwner	Verified	*The upgrade entry point reverts for any caller other than the owner.*	Report

ACCESS-COLLATERAL-RESCUE-01

Status: Verified	Only the owner can rescue ERC20 tokens held by the collateral token.

Rule Name	Status	Description	Link to rule report
rescueRevertsWithoutOwner	Verified	*The rescue entry point reverts for any caller other than the owner.*	Report

OracleAggregator Resolution Lifecycle

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

ORACLE-ARB-01

Status: Verified	Arbitrator hook failures never brick the lifecycle.

Rule Name	Status	Description	Link to rule report
thresholdDisputeEscalatesDespiteFailingHook	Verified	*A threshold-crossing dispute escalates even when the arbitrator's hook reverts.*	Report
reporterConflictEscalatesDespiteFailingHook	Verified	*A reporter conflict escalates even when the arbitrator's hook reverts.*	Report
adminResolveCompletesDespiteFailingHook	Verified	*Admin resolution during arbitration completes even when the arbitrator's hook reverts.*	Report

ORACLE-FIN-01

Status: Verified	The finalized outcome is caller-independent.

Rule Name	Status	Description	Link to rule report
finalizeOutcomeIsCallerIndependent	Verified	*Two successful finalizations of the same request from the same state deliver the same outcome, whoever calls and whatever array they submit.*	Report

finalizedOutcomeMatchesTheProposal	Verified	*A finalized result is exactly the one the standing proposal committed to.*	Report

ORACLE-LIFE-01

Status: Verified	Status is monotone with only three reachable transitions.

Rule Name	Status	Description	Link to rule report
activeIsNeverPersisted	Verified	*The Active status is never written to storage, so it cannot be observed.*	Report
statusIsMonotoneWithThreeTransitions	Verified	*Status never moves backwards, and only along the three reachable edges.*	Report

ORACLE-LIFE-02

Status: Verified	No resolution without quorum or authority.

Rule Name	Status	Description	Link to rule report
resolvedOnlyViaQuorumOrAuthority	Verified	*Every transition to Resolved is a quorum-backed finalize or an authorized override.*	Report

ORACLE-RES-01

Status: Verified	On an already-Resolved request, resolveResult is a no-op for any caller.

Rule Name	Status	Description	Link to rule report
resolvedResolveResultIsSilentNoOpForAnyCaller	Verified	*On a Resolved request, resolveResult no-ops silently for any caller, authorized or not.*	Report

ORACLE-TGT-01

Status: Verified	Settling a request writes a payout to exactly one market outcome, and that outcome's YES and NO shares always add up to denominator.

Rule Name	Status	Description	Link to rule report
payoutMappingBinaryAndIncremental	Verified	*Binary and incremental neg-risk reports write the complementary pair at the request's own condition index.*	Report
payoutMappingAtomic	Verified	*Atomic neg-risk reports the full denominator at the winner's condition index and zero elsewhere.*	Report

ORACLE-TGT-02

Status: Verified	Once a request has been settled, it can never be settled again.

Rule Name	Status	Description	Link to rule report
targetWritesOnlyFromFinalizeOrResolve	Verified	*No entry point other than finalize and resolveResult can ever write to the resolution target.*	Report

resolvedRequestCannotBeFinalizedAgain	Verified	*Once a request is Resolved, finalize always reverts.*	Report
arbitrationRequestedRequestCannotBeFinalized	Verified	*While a request is in arbitration, finalize always reverts.*	Report

OracleAggregator Vote Accounting

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

ORACLE-DISP-01			
Status: Verified		Dispute accounting and escalation at the threshold.	
Rule Name	Status	Description	Link to rule report
unregisteredRequestHasCleanDisputeState	Verified	*An unregistered request carries no dispute state.*	Report
disputeQuorumlmpliesEscalation	Verified	*While a request can still take disputes its dispute count is strictly below the threshold, so the threshold-crossing dispute escalates in the same transaction.*	Report
disputeAccountingAndEscalation	Verified	*A successful dispute counts once and escalates exactly at the threshold.*	Report
disputeRequiresPriorProposal	Verified	*A dispute can only follow a prior successful proposal.*	Report
disputeRepeatByTheSameModuleReverts	Verified	*A module cannot dispute twice on the same request.*	Report

ORACLE-THRESH-01	
Status: Verified	A live proposal is always backed by reporter quorum.

Rule Name	Status	Description	Link to rule report
proposalImpliesRegisteredRequest	Verified	*A standing proposal hash always exists in the config of a registered request.*	Report
thresholdBacksLiveProposal	Verified	*No outcome enters the dispute window unless the reporter threshold of distinct modules agreed.*	Report

ORACLE-VOTE-01

Status: Verified	Vote conservation across both counters.

Rule Name	Status	Description	Link to rule report
voteCountersAdvanceOnlyThroughReportOrDispute	Verified	*Both counters are monotone, flags are never cleared, and only the two vote entry points move them.*	Report

ORACLE-VOTE-02

Status: Verified	reportResult single-vote accounting.

Rule Name	Status	Description	Link to rule report
reportResultCastsExactlyOneVote	Verified	*A successful report sets exactly the caller's flag and increments exactly one vote key.*	Report
reportResultRepeatByTheSameModuleReverts	Verified	*A module cannot vote twice on the same request.*	Report

certora

ORACLE-WIND-01

Status: Verified	Window immutability: the dispute window is written exactly once, at proposal creation.

Rule Name	Status	Description	Link to rule report
windowIsFrozenOnceProposed	Verified	*Once a proposal stands, nothing moves its dispute window.*	Report
newProposalWindowIsNowPlusLiveness	Verified	*A fresh proposal's window is the current time plus the configured liveness.*	Report

ORACLE-WIND-02

Status: Verified	Window expiry closes reporting and disputing.

Rule Name	Status	Description	Link to rule report
reportRejectedAtOrAfterWindowEnd	Verified	*Reporting is rejected at and after the window end.*	Report
disputeRejectedAtOrAfterWindowEnd	Verified	*Disputing is rejected at and after the window end.*	Report

OracleAggregator Request Configuration

Module General Assumptions

- Loops are unrolled to at most 2 iterations.

Module Properties

ORACLE-CONF-02			
Status: Verified		A request's thresholds and result length are fixed at registration, and a registered request always has a non-zero arbitrator.	
Rule Name	Status	Description	Link to rule report
thresholdsOfRegisteredRequestNeverChange	Verified	*A registered request's thresholds never change.*	Report
registeredRequestHasAnArbitrator	Verified	*A registered request can never end up with a zero arbitrator.*	Report
initializeRequestConfiguresSingletonResultLength	Verified	*A request can only be registered with a result length of exactly 1.* *Assumption: At most one reporter module and one disputer module are registered by the call.*	Report
onlyInitializeRequestChangesResultLength	Verified	*No method other than registration ever moves a request's configured result length.*	Report

OracleAggregator Access Control

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

ORACLE-ACC-01

Status: Verified	resolveResult authority is confined to the configured arbitrator and admins.

Rule Name	Status	Description	Link to rule report
resolveResultAuthority	Verified	*A non-arbitrator, non-admin caller cannot resolve a request that is not yet Resolved.*	Report

ORACLE-ACC-02

Status: Verified	A non-zero finalizer is exclusive and no role bypasses it; the zero finalizer makes finalize permissionless.

Rule Name	Status	Description	Link to rule report
finalizerExclusivity	Verified	*A configured finalizer is exclusive, and no role bypasses the gate.*	Report
finalizeIsPermissionlessWhenFinalizerZero	Verified	*A zero finalizer makes finalize permissionless.*	Report

ORACLE-ACC-03

Status: Verified	reportResult is registered-reporter-only.

Rule Name	Status	Description	Link to rule report
reportResultRegisteredReporterOnly	Verified	*Only a registered reporter module for the request's event can report.*	Report

ORACLE-ACC-04

Status: Verified	disputeResult is registered-disputer-only.

Rule Name	Status	Description	Link to rule report
disputeResultRegisteredDisputerOnly	Verified	*Only a registered disputer module for the request's event can dispute.*	Report

ORACLE-ACC-05

Status: Verified	The config-mutation surface is operator-or-admin.

Rule Name	Status	Description	Link to rule report
configMutationRequiresOperatorOrAdmin	Verified	*Every operator-or-admin gated entry point reverts for a caller holding neither role.*	Report
initializeRequestIsOperatorOnly	Verified	*initializeRequest succeeds only for an operator caller.*	Report

certora

ORACLE-ACC-06

Status: Verified	Upgrades are owner-only.

Rule Name	Status	Description	Link to rule report
implementationChangesRequireOwner	Verified	*The ERC-1967 implementation slot only ever changes through the owner.*	Report

ORACLE-ACC-07

Status: Verified	Rules-role separation between the rule manager and the operator.

Rule Name	Status	Description	Link to rule report
productSpecWritesRequireRuleManagerOrAdmin	Verified	*Product-specification writes need the rule-manager or admin role.*	Report
requestRuleWritesRequireOperatorOrAdmin	Verified	*Per-request rule writes need the operator or admin role.*	Report
ruleManagerCannotWriteRequestRules	Verified	*A rule-manager-only account cannot write per-request rules.*	Report
operatorCannotWriteProductSpecs	Verified	*An operator-only account cannot write product specifications.*	Report

OOReporterModule Result Translation

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

OO-ATOMIC-01			
Status: Verified		The atomic neg-risk branch forwards a raw winner index, with an exact acceptance set.	
Rule Name	Status	Description	Link to rule report
atomicTranslationIsTheRawWinnerIndex	Verified	*On the atomic neg-risk branch the module forwards the length-one raw winner index, and that index is inside the event arity.*	Report
atomicAcceptsExactlyValidIndices	Verified	*The atomic branch accepts a settled price exactly when it is a non-negative whole multiple of the price unit whose quotient is below the event arity.*	Report

OO-PAYOUT-01			
Status: Verified		The price-to-payouts conversion is winner-takes-all and conserving.	
Rule Name	Status	Description	Link to rule report
payoutsPointwiseShape	Verified	*Multi-outcome conversion puts the full denominator in the winner slot and zero in every other slot.*	Report
payoutsSingleOutcomeShape	Verified	*Single-outcome conversion is total: it maps YES and the unresolvable price to their denominations and every other price to zero.*	Report

payoutsSumIsDenominator	Verified	*The converted vector sums to the result denominator for multi-outcome events, and to the mapped denomination for single-outcome ones.*	Report

OO-PRICE-01

Status: Verified	The binary price validator accepts a limited set of prices for every market type.

Rule Name	Status	Description	Link to rule report
validPriceExactSet	Verified	*The binary price validator accepts exactly the catalogued price set and nothing else.*	Report
p3AcceptedOnlyForBinary	Verified	*The unresolvable price is accepted under the binary market type and rejected under every other, so an incremental neg-risk subcondition cannot resolve to it.*	Report
negativePricesRejected	Verified	*Every negative price is rejected for every market type.*	Report

OO-RELAY-01

Status: Verified	Result integrity of the permissionless relay: the array the aggregator records is exactly the deterministic translation of the settled price, caller-independent.

Rule Name	Status	Description	Link to rule report
reportForwardsTheTranslationToTheAggregator	Verified	*A successful report makes the aggregator record exactly one vote, for the hash of the module's own translation, and for no other result.*	Report

relayIsCallerIndependent	Verified	*Two different callers relaying the same request from the same state make the aggregator record the same result.*	Report

OOReporterModule Registration and Access Control

Module General Assumptions

- Loops are unrolled to at most 3 iterations.

Module Properties

OO-ACCESS-01			
Status: Verified		Privileged state changes only by authorized writers.	
Rule Name	Status	Description	Link to rule report
registrationWritesRequireOperatorOrAggregator	Verified	*A registration appears only through createRequest by an operator, or through the aggregator batch.*	Report
aggregatorChangesRequireAdminOrInit	Verified	*The aggregator pointer changes only through an admin setAggregator or the one-shot initialize.*	Report
roleChangesRequireAuthorizedWriter	Verified	*Roles change only for the owner, for an admin, for a user renouncing their own roles, or in initialize.*	Report
implementationChangesRequireOwner	Verified	*The ERC-1967 implementation changes only for the owner.*	Report

OO-CUSTODY-01	
Status: Verified	No method moves ERC-20 value or ETH out of the module, and none grants an allowance.

Rule Name	Status	Description	Link to rule report
noValueLeavesTheModule	Verified	*No method moves ERC-20 value out of the module or grants an allowance on the module behalf.*	Report

OO-INIT-MONO-01

Status: Verified	A request registration never transitions from true back to false.

Rule Name	Status	Description	Link to rule report
requestInitializedIsMonotonic	Verified	*No method clears a registration: an initialized request never goes back to uninitialized.*	Report

OO-REG-01

Status: Verified	A requestId is registered exactly once globally across both entry points, and every registration in a successful batch belongs to the supplied event.

Rule Name	Status	Description	Link to rule report
createRequestIsOneShot	Verified	*createRequest cannot register a scope that is already registered.*	Report
batchCannotReRegisterAScope	Verified	*The aggregator batch cannot re-register a scope that is already registered.*	Report
batchRegistrationsBelongToTheSuppliedEvent	Verified	*Every scope a successful batch registered to belongs to the event the aggregator supplied.*	Report

CcipBridge Cross-Chain Fidelity

Module General Assumptions

- Loops are unrolled to at most 4 iterations.
- The legacy condition-id helpers are replaced by CVL models or over-approximated.
- Bridge mint and burn effects on the position manager are summarized with CVL models.
- The CCIP message send is handled by a CVL model acting as the router.

Module Properties

BRIDGE-01

Status: Verified after fix	A burn on the source chain is followed by an equivalent mint on the destination chain.

Rule Name	Status	Description	Link to rule report
bridgePositionsBurnsWhatItSends	Verified	*One message to the configured peer of the requested chain, carrying exactly the burned (recipient, positionIds, amounts) tuple; only the bridge's own balance is debited and no collateral moves.*	Report
bridgeCollateralBurnsWhatItSends	Verified	*One message to the configured peer carrying exactly (recipient, amount); collateral supply and the bridge's own balance both drop by that amount and positions are untouched.*	Report
bridgeResultMovesNoValue	Verified	*bridgeResult pushes resolution data only: exactly one message, and no mint or burn of positions or collateral.*	Report
receivePositionsMintsWhatWasSent	Verified	*A POSITIONS receive credits only the decoded recipient, by exactly the per-id amounts the payload carries, moves no collateral, and sends nothing onward.*	Report
receiveCollateralMintsWhatWasSent	Verified	*A COLLATERAL receive credits only the decoded recipient, by exactly the decoded amount, touches no positions, and sends nothing onward.*	Report

receiveAuth	Verified	*ccipReceive reverts unless the caller is the CCIP router and the decoded sender is the registered peer of the source chain selector.*	Report
sentPositionsAreDeliverable	Verified after fix	*Every accepted POSITIONS send executes on an honestly-configured destination, for an unconstrained recipient.*	Report
sentCollateralIsDeliverable	Verified after fix	*Every accepted COLLATERAL send executes on an honestly-configured destination, for an unconstrained recipient.*	Report

BRIDGE-ALLOWLIST-01

Status: Verified	A position whose module is not on the bridgeable allowlist is refused on both the send and the receive path.

Rule Name	Status	Description	Link to rule report
bridgeRejectsNonBridgeableModule	Verified	*bridgePositions reverts for a position whose module is not on the bridgeable allowlist.*	Report
receiveRejectsNonBridgeableModule	Verified	*Delivering a POSITIONS message whose module is not on the bridgeable allowlist reverts.*	Report

BRIDGE-RESULT-01

Status: Verified	A result leaves only the resolution chain, and is accepted only off it, only on its lane.

Rule Name	Status	Description	Link to rule report
resultExportOnlyFromResolutionChain	Verified	*bridgeResult reverts on every chain other than the resolution chain.*	Report
resultReceiveRejectedOnResolutionChain	Verified	*A RESULT message delivered to the chain that produced it reverts.*	Report
resultReceiveOnlyFromResolutionLane	Verified	*A RESULT message arriving on any lane other than the resolution chain's reverts.*	Report

Initialization Access Control

Module General Assumptions

- Loops are unrolled to at most 3 iterations.
- The legacy condition-id helpers are replaced by CVL models or over-approximated.

Module Properties

ACCESS-INIT-01

Status: Verified	Initializers run exactly once and implementations are permanently non-initializable, for each of PositionManager, CollateralToken, Exchange, OracleAggregator, OOReporterModule, BinaryModule, NegRiskModule, CombinatorialModule and CcipBridge.

Rule Name	Status	Description	Link to rule report
initVersionMovesOnlyFreshToOne (Initializable)	Verified	*The initialized version is frozen once non-zero, and the only transition any method can perform is 0 to 1.* *Assumption: multicall is excluded because delegatecall-to-self runs the same code on the same storage, making it equivalent to a sequence of direct calls.*	Report Report Report Report Report Report Report Report
initializeRequiresFresh (Initializable)	Verified	*initialize succeeds only from the fresh state and lands at exactly version 1.*	Report Report Report Report Report Report Report Report
initializeSucceedsAtMostOnce (Initializable)	Verified	*After a successful initialize, every further initialize reverts.*	Report Report Report Report Report Report Report Report

initVersionMovesOnlyFreshToOne (InitializableCombinatorial)	Verified	*The initialized version is frozen once non-zero, and the only transition any method can perform is 0 to 1.* *Assumption: multicall is excluded because delegatecall-to-self runs the same code on the same storage, making it equivalent to a sequence of direct calls.*	Report
initializeRequiresFresh (InitializableCombinatorial)	Verified	*initialize succeeds only from the fresh state and lands at exactly version 1.*	Report
initializeSucceedsAtMostOnce (InitializableCombinatorial)	Verified	*After a successful initialize, every further initialize reverts.*	Report

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.